

LeetCode 730: Count Different Palindromic Subsequences

Authored by: Arpit Raj, LNMIIT Jaipur

Date: September 5, 2026

Atlassian | Goldman Sachs

1. Problem Overview & Example

Example: *b c c b*

Palindromic subsequences: b, c, bccb, bcb, cc, bb

Total count = 6

$$\begin{bmatrix} \mathbf{1} & \mathbf{r} \end{bmatrix}$$

2. Core DP Transitions

Condition 1: If $s[l] \neq s[r]$

$$dp[l][r] = dp[l+1][r] + dp[l][r-1] - dp[l+1][r-1]$$

Explanation: Need to subtract commons.

Condition 2: Say $s[l] == s[r]$

Then what we can do is take any palindrome inside $[l+1 \dots r-1]$ and add those $s[l], s[r]$ (l and r).

Example: a [...] a

Now inside can be 'b' or 'bb'.

New palindromes = aba and abba

Now say inside had 3 palindromes P_1, P_2, P_3 .

Then we get: $P_1, P_2, P_3, a+P_1+a, a+P_2+a, a+P_3+a, a, aa$.

Hence $2 \times dp[l+1][r-1] + 2$

CATCH! (HANDLING DUPLICATES INSIDE 'A...A')

Case 1: If there is **no 'a'** inside, then we do:

$$2 * dp[l+1][r-1] + 2$$

Case 2: But if there is exactly **one 'a'** inside, then 'aa' is duplicate so we do:

$$2 * dp[l+1][r-1] + 1$$

Case 3: Case when there are ≥ 2 **a's** inside:

a [a a] a
 ↑ ↑
 left right

Then $dp[left+1][right-1]$ are getting counted twice. Hence:

$$2 * dp[l+1][r-1] - dp[left+1][right-1]$$

3. C++ Implementation

```
class Solution {
public:
    const int MOD = 1e9 + 7;

    int countPalindromicSubsequences(string s) {
        int n = s.size();
        vector<vector<int>> dp(n, vector<int>(n, 0));

        for(int i = 0; i < n; i++) {
            dp[i][i] = 1;
        }

        for(int len = 2; len <= n; len++) {
            for(int l = 0; l + len - 1 < n; l++) {
                int r = l + len - 1;

                if(s[l] != s[r]) {
                    dp[l][r] = dp[l+1][r] + dp[l][r-1] - dp[l+1][r-1];
                }
                else {
                    // a.....a
                    // beech me k palindrome khojo

                    // case 1: a.....a
                    // k original, k after wrapping a...a around, a and aa (2)

                    // case 2: a.....a.....a
                    // aa duplicate hence -1

                    // case 3: a....a...a....a
                    // [left+1][right-1] waale counted twice

                    int left = l + 1;
                    int right = r - 1;

                    while(left <= right && s[left] != s[l]) {
                        left++;
                    }

                    while(left <= right && s[right] != s[r]) {
                        right--;
                    }

                    if(left > right)
                        dp[l][r] = 2 * dp[l+1][r-1] + 2;
                    else if(left == right)
                        dp[l][r] = 2 * dp[l+1][r-1] + 1;
                    else

```

```
        dp[l][r] = 2 * dp[l+1][r-1] - dp[left+1][right-1];
    }

    dp[l][r] %= MOD;
    if(dp[l][r] < 0) {
        dp[l][r] += MOD;
    }
}

return dp[0][n-1];
};
```